

ML MODEL FOR FITSYNC

```
%pip install pandas
```

```
import pandas as pd
```

```
# Load the CSV file
```

```
df = pd.read_csv('new_heart_rates.csv')
```

```
# Display the first few rows of the dataframe
```

```
df.head()
```

```
df["heart_rates"] = df["heart_rates"].apply(eval)
```

```
df.head()
```

```
%pip install scikit-learn
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
df["heart_rates"]
```

```
%pip install numpy
```

```
import numpy as np
```

```

scalers = {}

# Function to normalize heart rate lists
def normalize_user_data(user_df):
    scaler = MinMaxScaler()

    # Flatten the heart rate data for the user
    all_heart_rates = np.concatenate(user_df["heart_rates"].values).reshape(-1, 1)

    # Fit and transform the data
    normalized_hr = scaler.fit_transform(all_heart_rates).flatten()

    # Store the scaler for later use
    scalers[user_df["user_id"].iloc[0]] = scaler

    # Split back the normalized values into original lists
    split_indices = np.cumsum([len(l) for l in user_df["heart_rates"].values])[:-1]
    normalized_lists = np.split(normalized_hr, split_indices)

    return pd.Series(normalized_lists, index=user_df.index)

```

```

df["normalized_heart_rates"] = df.groupby("user_id", group_keys=False).apply(n

```

```

df.head()

```

```

# Define sequence length (e.g., 50 time steps per input)
SEQ_LENGTH = 50

# Function to generate sequences from a continuous list of heart rates
def create_sequences(hr_list, seq_length):
    sequences = []
    for i in range(len(hr_list) - seq_length + 1):
        sequences.append(hr_list[i : i + seq_length]) # Extract window

```

```

    return np.array(sequences)

# Create sequences per user and stack
X_train = []

for _, user_df in df.groupby("user_id"):
    user_hr = np.concatenate(user_df["normalized_heart_rates"].values) # Flatten
    user_sequences = create_sequences(user_hr, SEQ_LENGTH)
    X_train.append(user_sequences)

X_train = np.vstack(X_train) # Combine all users
X_train = np.expand_dims(X_train, axis=-1) # Add feature dimension for LSTM (b

print(f"Final shape of X_train: {X_train.shape}") # Expected: (num_samples, SEQ

```

```
%pip install torch
```

```

import torch
import torch.nn as nn
import torch.optim as optim

# Define LSTM Autoencoder
class LSTMAutoencoder(nn.Module):
    def __init__(self, seq_len, n_features, latent_dim=32):
        super(LSTMAutoencoder, self).__init__()
        self.encoder = nn.LSTM(input_size=n_features, hidden_size=64, num_layers=1)
        self.latent = nn.Linear(64, latent_dim) # Bottleneck layer
        self.decoder = nn.LSTM(input_size=latent_dim, hidden_size=64, num_layers=1)
        self.output_layer = nn.Linear(64, n_features)

    def forward(self, x):
        # Encoder
        encoded, _ = self.encoder(x)
        latent = self.latent(encoded[:, -1, :]) # Take last time step output

```

```
# Repeat latent representation for each time step in decoder
latent_repeated = latent.unsqueeze(1).repeat(1, x.shape[1], 1)
```

```
# Decoder
decoded, _ = self.decoder(latent_repeated)
reconstructed = self.output_layer(decoded)
```

```
return reconstructed
```

```
# Define model
seq_len = 50 # Time steps
n_features = 1 # Single feature (heart rate)
model = LSTMAutoencoder(seq_len, n_features)
```

```
from torch.utils.data import DataLoader, TensorDataset
```

```
# Convert NumPy array to PyTorch Tensor
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
```

```
# Create DataLoader for efficient batching
train_dataset = TensorDataset(X_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=256, shuffle=True)
```

```
# Define loss function & optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
# Move model to GPU if available
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
model.to(device)
```

```
device
```

```

num_epochs = 5
for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0
    print(f"Epoch {epoch+1} {len(train_loader)}")
    i = 0
    for batch in train_loader:
        batch = batch[0].to(device)
        optimizer.zero_grad()
        reconstructed = model(batch)
        loss = criterion(reconstructed, batch) # Compare reconstruction with origin
        loss.backward()
        optimizer.step()
        print(f"epoch {epoch+1} batch {i+1}")
        epoch_loss += loss.item()
        i += 1

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss / len(train_loader)}")

```

```

torch.save(model.state_dict(), "heart_rate_predictor_lstm_model.pth")

```

```

%pip install torch.onnx

```

```

import torch.onnx

```

```

dummy_input = torch.randn(1, 50, 1).to(device) # (batch_size=1, seq_len=50, feat_dim=1)
torch.onnx.export(model, dummy_input, "lstm_model.onnx",
                  export_params=True, input_names=["input"], output_names=["output"])

```

Details of the ML model - LSTM Based Autoencoder: